Отчет по лабораторной работе 7

Программа mpi_array_sum.c считает $S = \Sigma\, a_i$, $a_i = i + 1$, $i = 0 \ldots n-1$. Процесс 0 читает n из аргумента и рассылает его через MPI_Bcast (вместе с флагом режима); все процессы одинаково проверяют $n > 0$ и $n \bmod p = 0$ и при ошибке вместе завершаются через MPI_Finalize без зависания. Процесс 0 создаёт массив, MPI_Scatter раздаёт по n/p подряд идущих элементов, каждый процесс считает локальную сумму S_r (long long, MPI_LONG_LONG_INT). Затем MPI_Gather собирает все S_r на процесс 0, где они складываются циклом; MPI_Reduce(MPI_SUM) даёт S только процессу 0; MPI_Allreduce(MPI_SUM) даёт S всем процессам. Процесс 0 сравнивает все три результата с $n(n + 1)/2$.

Доп. задания: при флаге scatterv массив делится через MPI_Scatterv на части n/p и n/p + 1 (counts/displs), так что n может не делиться на p. Второе доп. задание: S нужна всем — каждый процесс по S из Allreduce находит среднее S/n и сам считает вклад своих элементов в дисперсию $\Sigma(a_i - S/n)^2$; итог сверяется с $(n^2 - 1)/12$ (отн. ошибка $\le 10^{-12}$), а MPI_Reduce(MPI_LAND) подтверждает, что S верна на каждом процессе. С Reduce для этого понадобился бы ещё и Bcast, Allreduce делает обе операции за один вызов.

Команды: mpicc -O2 -Wall -Wextra mpi_array_sum.c -o mpi_array_sum; mpiexec -n 4 ./mpi_array_sum 1000000; mpiexec -n 6 ./mpi_array_sum 1000000 scatterv; mpiexec --oversubscribe -n 8 ./mpi_array_sum 1000000. Система: i5-9400F (6 ядер), WSL2 Ubuntu 26.04, Open MPI 5.0.10, gcc 15.2.

Замер: время конвейера Scatter + локальная сумма + Gather + Reduce + Allreduce (создание массива не входит): максимум по процессам (MPI_Reduce MAX), среднее по 10 повторам после прогрева, медиана 3 запусков.

3.1. Корректность (последние две строки — MPI_Scatterv, n не делится на p или выбран режим scatterv)

p	n	Ожидаемая	Gather	Reduce	Allreduce	Корректно?
2	100	5050	5050	5050	5050	да
4	100	5050	5050	5050	5050	да
4	10000	50005000	50005000	50005000	50005000	да
8	1000000	500000500000	500000500000	500000500000	500000500000	да
4	10	55	55	55	55	да
6	1000000	500000500000	500000500000	500000500000	500000500000	да

3.2. Размер массива, p = 4

n	Время, с	Верно?
10000	$1.28 \cdot 10^{-5}$	да
100000	$3.91 \cdot 10^{-5}$	да
1000000	$3.52 \cdot 10^{-4}$	да
10000000	$5.71 \cdot 10^{-3}$	да

3.3. Число процессов, $n = 10^6$

p	Время, с	S_p	E_p
1	$5.72 \cdot 10^{-4}$	1.00	1.00
2	$5.00 \cdot 10^{-4}$	1.14	0.57
3[v]	$4.63 \cdot 10^{-4}$	1.24	0.41
4	$3.65 \cdot 10^{-4}$	1.57	0.39
6[v]	$5.33 \cdot 10^{-4}$	1.07	0.18
8*	$3.63 \cdot 10^{-4}$	1.58	0.20

3.4. Агрегирование, мкс (20 000 повт.)

p	Gather + сумма	Reduce	Верно?
2	0.11	0.14	да
4	0.50	0.32	да
6	0.65	0.32	да
8*	1.55	1.67	да

[v] — MPI_Scatterv (10^6 не делится на 3 и 6); * — 8 процессов на 6 ядрах (переподписка).

Вывод:

1) Все три способа дают точную сумму $n(n + 1)/2$ при всех p и n, включая неравные части через Scatterv. 2) Различие операций: MPI_Gather только собирает значения — процесс 0 получает массив из p локальных сумм и складывает их сам; MPI_Reduce собирает и сразу объединяет (MPI_SUM) внутри MPI, результат только у root; MPI_Allreduce — то же, но результат у всех процессов (≈ Reduce + Bcast за один вызов). Gather нужен, когда важны отдельные значения; Reduce — когда итог нужен одному процессу; Allreduce — когда итог нужен всем для продолжения счёта. Код с Reduce/Allreduce короче: нет буфера на p элементов и ручного цикла. 3) Время растёт почти линейно по n (с 10^6 до 10^7 — в 16 раз: 40 МБ массива уже не помещаются в L3), основная доля — MPI_Scatter: процесс 0 копирует весь массив другим процессам, а само суммирование дешевле копирования. 4) Поэтому ускорение по p мало (≤ 1.6) и далеко от p: при росте p уменьшается только локальная сумма, а объём рассылки остаётся n элементов; при p = 8 добавляется переподписка. Если бы данные уже лежали на процессах, ускорение было бы ближе к линейному. 5) Одна операция агрегирования занимает 0.1–2 мкс; при p = 2 Gather чуть быстрее, при p = 4–6 Reduce быстрее на 30–50 % (сумма считается по дереву, а не последовательно на root), при p = 8 разница в пределах шума. Делать вывод по одному замеру нельзя — разброс запусков доходит до 2 раз.